

Assignment 4 – Home assignment: Modelling a nonlinear pendulum

Subject: Intelligent Systems in Decision Support

Goal of the assignment

The goal of this assignment is to model a nonlinear pendulum and to compare several identification approaches on the same plant. The resulting model will later be used as the internal model of a predictive controller, so it should be accurate enough to predict the pendulum response several steps ahead. In this assignment we focus only on data generation and identification.

The pendulum

The plant is a single rigid pendulum rotating in the vertical plane about a fixed pivot. The input is the applied torque $u(t)$ and the (measured) output is the angle $\varphi(t)$, measured from the lower vertical equilibrium. Its dynamics are governed by

$$J \ddot{\varphi}(t) = u(t) - mgL \sin(\varphi(t)) - c \dot{\varphi}(t) - \tau_f(\dot{\varphi}(t)),$$

where J is the moment of inertia, mgL the gravitational restoring term, $c\dot{\varphi}$ viscous friction, and τ_f a stick-slip (Coulomb) friction term with static and kinetic components. The system is therefore mildly nonlinear in two distinct ways: the sinusoidal gravity term and the discontinuous friction at $\dot{\varphi} = 0$.

The simulator additionally enforces hard mechanical end-stops at $\varphi_{\min} = 10^\circ$ and $\varphi_{\max} = 100^\circ$ (with partial restitution at contact), adds independent Gaussian measurement noise on the angle, and quantises the measurement with an analog-to-digital converter of resolution 0.01 rad. The file `pendulum_process_obf.p` contains the obfuscated simulator that you will use to generate the data.

It is up to you to choose an appropriate sampling strategy, identify the models listed below, and then evaluate them. The models must be accurate on the operating range $\varphi \in [20^\circ, 80^\circ]$.

Home assignment

Your task is to model the pendulum described above using several different methods and to compare them on the same plant.

Identify system using following models:

- **Previous-sample model:** A simple baseline model that assumes the output stays equal to the last measured value.

$$\hat{y}(k+N) = y(k)$$

Why: serves as a sanity check for your implementation and choice of model. If your model cannot beat this simple baseline, it is likely that there is a bug in your code or that your model is not appropriate for the data.

- **Linear model:** Identify a linear model of the system.
- **Fuzzy Takagi-Sugeno model, local identification:** Identify a Takagi-Sugeno model of the system using local identification
- **Fuzzy Takagi-Sugeno model, global identification:** Identify a Takagi-Sugeno model of the system using global identification
- **Neural network:** Identify a feedforward neural network model of the system.

Identify each model and evaluate it appropriately. Your choices of how to generate the identification data, of hyperparameters, of the evaluation criterion, and of how to compare the models all matter and will be graded.

Grading criteria

- The main criterion is the quality of the model on the validation signal.
- In addition to the already published criteria for the reports, I will evaluate the following criteria:
 - Sampling strategy (whether the chosen sampling strategy leads to a better model or not).
 - Choice of model hyperparameter / algorithm
 - Model-evaluation procedure (choice of criterion, comparison with other methods, e.g. neural networks / linear model / previous-sample model).

Using the simulator

The function `pendulum_process_obf(u, Ts)` simulates the response of the pendulum. Since it is a process simulation, the function keeps an internal state (angle and angular velocity of the pendulum).

- **Resetting the process:** To reset the process to its initial state, call the function with an empty input:

`pendulum_process_obf([], 0)`

- **Simulation:**

`y = pendulum_process_obf(u, Ts)`

- **u:** Input torque (Nm). May be a scalar (for a single simulation step) or a vector (for several steps).
- **Ts:** Sampling time (s). If it is a scalar, the same time step is used for all samples. If a vector of times is given, it specifies the instants at which the input **u** changes.
- **y:** Output of the system, a vector of length N containing the measured angle in radians.

An example of how to use the function can be found in the file `example_pendulum_process.m`. Note: many settings in the example are deliberately suboptimal and serve only as a syntax demonstration — do not treat them as a reference design.